

AccessHub Platform Implementation Plan

Complete Technical Specification for Angular SPA & Node.js Role-Based Access Portal

Frameworks: Angular 16+ & Express.js

Database: MongoDB (Local)

Security: JWT & RBAC Middleware

Date: October 2026

Executive Summary

AccessHub is a production-style single page application designed for enterprise applicant verification records management. It enforces strict Role-Based Access Control (RBAC) across client views, routing, and backend REST APIs. The architecture implements modular Angular components, reactive forms, RxJS observables, artificial latency simulation, and automated test coverage.

01. Technology Stack & Infrastructure

LAYER	TECHNOLOGY	PURPOSE & KEY FEATURES
Frontend SPA	Angular 16, TypeScript, SCSS	Modular lazy-loaded architecture, Reactive Forms, Route Guards, HTTP Interceptors.
Reactive Layer	RxJS 7+	BehaviorSubjects for auth state, forkJoin for parallel API aggregation, debounceTime for search.
Backend API	Node.js, Express.js (TypeScript)	RESTful API layer, route-level JWT & RBAC middleware, async latency simulator.
Database	MongoDB (Mongoose ODM)	Local persistent database (`mongodb://localhost:27017/accesshub`), schema validation, bcrypt salting.
API Specs	Swagger / OpenAPI 3.0	Interactive API testing & schema definition at `/api-docs`.
Testing	Jest & Supertest	14 automated integration tests covering authentication, RBAC authorization, and API routes.

02. Backend Architecture & REST API Design

Database Models (MongoDB)

- User Model:** Stores `userId`, `name`, `email`, `password` (hashed with salt), `role` (`ADMIN` | `GENERAL\_USER`), `status` (`ACTIVE` | `INACTIVE`).
- Record Model:** Stores `recordId`, `title`, `category`, `status`, `owner`, `priority`, and `description`.

Security & Middleware

- JWT Auth:** Validates `Bearer` tokens on all private endpoints.
- requireAdmin:** Restricts user directory & global management to administrators.
- Delay Simulator:** Handles query parameter `?delay=ms` to simulate latency.

METHOD	ENDPOINT	ACCESS	DESCRIPTION
POST	/api/auth/login	Public	Validates user ID, password, and active status; issues JWT token.
GET	/api/records	Authenticated	Returns records filtered by ownership for General Users, or all for Admin.
POST	/api/records	Authenticated	Creates a new verification record tagged with current user ownership.
PUT / DELETE	/api/records/:id	Owner / Admin	Modifies or removes a record (RBAC checked).
GET	/api/users	Admin Only	Retrieves full user management directory with status and roles.
PATCH	/api/users/:id/status	Admin Only	Toggles account status between ACTIVE and INACTIVE.

03. Frontend Architecture & Angular Modules

Core & Shared Modules

- AppRoutingModule:** Shell wrapper using ``LayoutComponent`` for authenticated routes.
- AuthGuard:** Redirects unauthenticated visits to ``/login``.
- RoleGuard:** Enforces ``data: { role: 'ADMIN' }`` protection.
- AuthInterceptor:** Injects ``Authorization: Bearer ``.
- Shared Components:** Reusable Spinner, Badges, and Dialogs.

Feature Modules (Lazy Loaded)

- AuthModule:** Two-panel login UI with credential quick-fill.
- DashboardModule:** Key stat cards and recent records via ``forkJoin``.
- RecordsModule:** Table with debounce search, filters, and latency toggle.
- AdminModule:** User directory with modal creation and status toggles.
- ProfileModule:** Profile updates, password changes, and RBAC matrix.

⚡ **Async Delay Simulation Feature:** Both the Records and User Management views feature an interactive latency selector bar (0ms, 1000ms, 2000ms, 3000ms). When toggled, the frontend requests data with an artificial server delay, triggering clean loading spinner states and demonstrating async handling.

04. Role-Based Access Control (RBAC) Matrix

APPLICATION CAPABILITY	ADMINISTRATOR (ADMIN)	GENERAL USER (GENERAL_USER)	ENFORCEMENT LAYER
Platform Dashboard	FULL PLATFORM METRICS	PERSONAL STATS ONLY	Backend aggregation & UI view filtering
Verification Records	ALL RECORDS (CROSS-USER)	OWN RECORDS ONLY	Backend <code>`owner`</code> query filter
Create Records	ALLOWED	ALLOWED	Reactive Form validation
Edit / Delete Records	ANY RECORD	OWN RECORDS ONLY	Backend RBAC ownership check
User Directory (<code>`/admin/users`</code>)	FULL ACCESS	FORBIDDEN (BLOCKED)	Angular <code>`RoleGuard`</code> & Express <code>`requireAdmin`</code>
Toggle Account Status	ALLOWED	FORBIDDEN	Express <code>`requireAdmin`</code> middleware
Edit Profile & Password	SELF ONLY	SELF ONLY	JWT user claims

05. Quality Assurance & Test Verification

Integration Test Suite (14/14 Passed)

- ✓ Admin & User login validation
- ✓ Password hashing and verification
- ✓ Role matching & rejection on invalid roles
- ✓ 403 Forbidden on unauthorized user directory access
- ✓ 401 Unauthorized on missing JWT tokens
- ✓ Ownership-based record isolation

Live Port Verification

- Frontend:** `http://localhost:4200` (HTTP 200 OK)
- Backend:** `http://localhost:5000` (HTTP 200 OK)
- Swagger UI:** `http://localhost:5000/api-docs`
- Database:** MongoDB on `27017`
- TypeScript:** 0 errors (`tsc --noEmit` clean)

06. Pre-seeded Demo Credentials

ROLE	USER ID	PASSWORD	ACCOUNT STATUS	DESCRIPTION
Administrator	admin	admin123	ACTIVE	Superuser account with full access to records and user directory.
General User	user	user123	ACTIVE	Standard user account with 3 pre-seeded personal verification records.
General User	john	user123	ACTIVE	Employee account with HR & IT setup records.
General User	sarah	user123	ACTIVE	Compliance officer account with legal & finance records.
Inactive User	emily	user123	INACTIVE	Demonstrates inactive account protection (login blocked with 403).